

Physics-Informed Fluid Dynamics Simulation with Simplicial Neural Networks and Kolmogorov–Arnold Networks

PI-EKAN: A Physics-Informed Equivariant KAN Surrogate
for Mesh-Based Flow Prediction

Shreelakshmi Somayagi

Dept. of CSE (AI)

PES University, Bengaluru, India

pes1ug23am290@pesu.pes.edu

A Amrith

Dept. of CSE (AI)

PES University, Bengaluru, India

pes1ug23am045@pesu.pes.edu

Sanat Shirwaicar

Dept. of CSE (AI)

PES University, Bengaluru, India

pes1ug23am266@pesu.pes.edu

Anirudh Suraj

Dept. of CSE (AI)

PES University, Bengaluru, India

pes1ug23am050@pesu.pes.edu

Dr. Bhaskarjyoti Das

Dept. of CSE (AI)

PES University, Bengaluru, India

bhaskarjyoti01@gmail.com

Abstract—Simulating fluid dynamics at scale remains one of the central challenges of computational science, with classical solvers often trading accuracy for speed in an unsatisfying compromise. In this work we introduce PI-EKAN v9, a physics-informed surrogate model that learns to advance unstructured-mesh fluid states in time by combining three complementary ideas: *simplicial neural networks* that propagate information across nodes, edges, and triangular faces of a Delaunay triangulation; *Kolmogorov–Arnold Networks (KANs)* as the nonlinear building blocks inside every message-passing step; and a rich physics-informed loss that enforces incompressibility via a finite-volume divergence operator derived directly from the FEM shape functions. Training stability over long autoregressive rollouts is achieved through Truncated Backpropagation Through Time (TBPTT) with a rollout-consistency term, noise annealing, and a tanh-gated output clamp that prevents exponential error growth without hurting one-step accuracy. A conformal prediction wrapper finally turns the deterministic output into a statistically valid 90% coverage guarantee on the incompressibility violation. Experiments on the EAGLE benchmark show that PI-EKAN consistently outperforms a parameter-matched MLP baseline on multi-step rollout quality (winning 25–30 of 30 future steps) while remaining within a comparable parameter budget and providing certified uncertainty bounds.

I. INTRODUCTION

Computational fluid dynamics (CFD) sits at the heart of engineering design, climate modelling, and fundamental physics research, yet traditional solvers scale poorly: refining the spatial mesh by a factor of two roughly octuples the computational cost in three dimensions. The last few years have seen a surge of interest in data-driven *surrogate* models that replace or accelerate the inner simulation loop after a one-time, offline training phase. The appeal is clear – once trained, a neural surrogate can predict a future state in milliseconds – but the pitfalls are equally clear: surrogates trained purely

on data can drift dramatically when rolled out beyond their training horizon, and they offer no guarantees about physical plausibility.

Several promising directions have emerged. Graph neural networks (GNNs) [11, 9] adapt naturally to unstructured meshes by treating mesh nodes as graph vertices and edges as communication channels. Physics-informed neural networks (PINNs) [12] embed the governing PDEs directly into the loss function. Higher-order topological methods [4] extend GNNs to operate on edges and faces as well, capturing the circulation and flux structure that classical node-only message passing misses. Most recently, Kolmogorov–Arnold Networks (KANs) [10] have been proposed as a drop-in replacement for multi-layer perceptrons that learn their activation functions from data, offering richer expressivity per parameter.

This paper brings these threads together. We describe the design, implementation, and evaluation of PI-EKAN, a surrogate model for two-dimensional incompressible flow that is simultaneously:

- 1) *topologically aware* – operating on the full 0-, 1-, and 2-chain complex of the Delaunay mesh;
- 2) *equivariant* – using geometric edge features (unit tangent, edge length, speed difference) that are invariant to global translation;
- 3) *physics-constrained* – penalising divergence via a FEM-exact finite-volume operator during training;
- 4) *stable over long horizons* – trained with TBPTT, a rollout consistency loss, and a saturating output clamp; and
- 5) *certifiably uncertain* – wrapped in a split conformal prediction procedure that provides a coverage guarantee on the per-face mass-conservation error.

The result is a single, self-contained model that can be dropped onto any Kaggle- or Colab-style GPU and trained end-to-end in a few hours.

A. Paper Organisation

Section II reviews the necessary background. Section III describes the architecture, topology, and training strategy in detail. Section IV covers the composite loss function. Section V describes the conformal prediction wrapper. Section VI presents experiments and ablations. Section ?? discusses limitations and future work.

II. BACKGROUND

A. Simplicial Complexes and Hodge Theory

A *simplicial complex* $\mathcal{K}$ is a collection of simplices – points (0-simplices), edges (1-simplices), and triangles (2-simplices) – closed under the face relation. For a Delaunay triangulation of V mesh nodes one obtains a complex with V vertices, E edges, and F triangular faces. The algebraic topology of $\mathcal{K}$ is captured by the *boundary maps*

$$\mathbf{B}_1 \in \mathbb{R}^{V \times E}, \quad \mathbf{B}_2 \in \mathbb{R}^{E \times F}, \quad (1)$$

where $(\mathbf{B}_1)_{v,e} = \pm 1$ if node v is the head (resp. tail) of edge e and zero otherwise, and $(\mathbf{B}_2)_{e,f} = \pm 1$ if edge e is a positively (resp. negatively) oriented boundary edge of face f .

The boundary maps satisfy $\mathbf{B}_1 \mathbf{B}_2 = 0$, the fundamental identity of algebraic topology. The *Hodge Laplacians* on nodes, edges, and faces are

$$\mathcal{L}_0 = \mathbf{B}_1 \mathbf{B}_1^\top, \quad (2)$$

$$\mathcal{L}_1 = \mathbf{B}_1^\top \mathbf{B}_1 + \mathbf{B}_2 \mathbf{B}_2^\top, \quad (3)$$

$$\mathcal{L}_2 = \mathbf{B}_2^\top \mathbf{B}_2. \quad (4)$$

Intuitively, $\mathcal{L}_0$ is the ordinary graph Laplacian, $\mathcal{L}_1$ mixes *gradient* information (how node values differ across an edge) with *curl* information (how edge values circulate around a face), and $\mathcal{L}_2$ captures the divergence of flux through faces. Simplicial Neural Networks [3, 13] replace classical graph convolution with message passing that respects this Hodge structure.

B. Kolmogorov–Arnold Networks

The universal approximation theorem guarantees that any continuous function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ can be approximated by an MLP, but the approximation may require exponentially many neurons. The Kolmogorov–Arnold representation theorem [7] gives a tighter decomposition:

$$f(x_1, \dots, x_n) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right), \quad (5)$$

where both the inner functions $\phi_{q,p}$ and the outer functions Φ_q are continuous and learned. Liu et al. [10] operationalise this as the KAN architecture: each *connection* carries a learnable B-spline activation, and no fixed nonlinearity (ReLU, SiLU) is needed. The grid-based splines have a natural width parameter (grid size) and an order parameter (spline order, typically 3)

that jointly control expressivity. In practice KANs achieve comparable or better accuracy to MLPs on physics problems while using fewer effective parameters [10], making them attractive for surrogate modelling.

III. METHOD: THE PI-EKAN ARCHITECTURE

A. Mesh Construction and Simplicial Operators

Given a point cloud of mesh nodes $\{x_v\}_{v=1}^V \subset \mathbb{R}^2$, we compute the Delaunay triangulation, which maximises the minimum angle and is known to produce well-conditioned finite elements. This yields the vertex set, edge list, and face list needed to instantiate the boundary operators $\mathbf{B}_1$ and $\mathbf{B}_2$. We pre-compute and cache (as sparse COO tensors) the following operators:

TABLE I
PRE-COMPUTED TOPOLOGICAL OPERATORS

Symbol	Shape	Meaning
$\mathbf{B}_1$	$V \times E$	Node–edge incidence (signed)
$\mathbf{B}_1^\top$	$E \times V$	Edge–node incidence
$\mathbf{B}_2$	$E \times F$	Edge–face incidence (signed)
$\mathbf{B}_2^\top$	$F \times E$	Face–edge incidence
$\mathcal{L}_0, \mathcal{L}_1, \mathcal{L}_2$	square	Hodge Laplacians
$\mathbf{D}_{\text{div}}$	$F \times 2V$	FEM-exact divergence
$\mathbf{W}_{\text{f2n}}$	$V \times F$	Face-to-node averaging

All operators are stored as PyTorch sparse COO tensors and moved to the GPU alongside the model. Degree vectors $\mathbf{d}_0 \in \mathbb{R}^V$, $\mathbf{d}_1 \in \mathbb{R}^E$, $\mathbf{d}_2^{B_1^\top} \in \mathbb{R}^E$, $\mathbf{d}_2^{B_2} \in \mathbb{R}^E$ are precomputed for normalised aggregation.

B. Feature Engineering

a) *Node features*: At time t each node v carries a feature vector

$$\mathbf{h}_v^{(t)} = [\mathbf{u}_v^{t-k:t}, p_v^{(t)}, (1-\phi_v), \omega_v^{(t)}, (\nabla p)_v^{x,(t)}, (\nabla p)_v^{y,(t)}, \|\mathbf{u}_v^{(t)}\|] \in \mathbb{R}^{d_n}. \quad (6)$$

where $\mathbf{u}_v^{t-k:t}$ stacks the velocity vectors at the previous $k = 2$ timesteps (giving $2k = 4$ scalars), $p_v^{(t)}$ is normalised pressure, $\phi_v \in \{0, 1\}$ indicates whether node v is a solid boundary, and the remaining enrichment terms encode the local vorticity ω_v (aggregated from edge circulations via $|\mathbf{B}_1|$), pressure gradient components (signed edge pressure differences projected along edge unit tangents), and local flow speed.

b) *Edge features*: Each directed edge ($u \rightarrow v$) carries

$$\mathbf{e}_{uv}^{(t)} = [\ell_{uv}, \hat{\mathbf{e}}_{uv}, \|\|\mathbf{u}_v\| - \|\mathbf{u}_u\|\|] \in \mathbb{R}^4, \quad (7)$$

where $\ell_{uv} = \|x_v - x_u\|$ is the physical edge length (a scale cue), $\hat{\mathbf{e}}_{uv}$ is the unit tangent vector (rotation-invariant direction cue), and the last term captures the magnitude of the speed gradient across the edge, which is a cheap proxy for strain rate.

C. The Three-Level Message-Passing Encoder

At the core of PI-EKAN are $L = 3$ stacked *simplicial message-passing blocks*, each of which updates representations at all three chain levels simultaneously.

a) *Node update*: For block ℓ , node v aggregates messages from its incident edges and from the direct node graph Laplacian:

$$\mathbf{x}_v^{(\ell+1)} = \mathbf{x}_v^{(\ell)} + \Phi_n^{(\ell)} \left(\mathbf{x}_v^{(\ell)}, \frac{1}{d_v} \sum_{e \ni v} \mathbf{x}_e^{(\ell)}, \frac{1}{d_v} \sum_{v' \sim v} \mathbf{x}_{v'}^{(\ell)} \right), \quad (8)$$

where $\Phi_n^{(\ell)}$ is a KAN block (or MLP fallback) and d_v is the node degree.

b) *Edge update*: Each edge e aggregates from its two endpoint nodes (gradient signal via $\mathbf{B}_1^\top$), from the surrounding faces (curl signal via $\mathbf{B}_2$), and from the adjacent edges (harmonic signal via $\mathcal{L}_1$):

$$\mathbf{x}_e^{(\ell+1)} = \mathbf{x}_e^{(\ell)} + \Phi_e^{(\ell)} \left(\mathbf{x}_e^{(\ell)}, \frac{1}{d_e^{\mathbf{B}_1}} \mathbf{B}_1^\top \mathbf{X}_n^{(\ell)}, \frac{1}{d_e^{\mathbf{B}_2}} \mathbf{B}_2 \mathbf{X}_f^{(\ell)} \right). \quad (9)$$

c) *Face update*: Face features $\mathbf{x}_f^{(\ell)}$ are updated from the bounding edges via $\mathbf{B}_2^\top$ and from adjacent faces via $\mathcal{L}_2$:

$$\mathbf{x}_f^{(\ell+1)} = \mathbf{x}_f^{(\ell)} + \Phi_f^{(\ell)} \left(\mathbf{x}_f^{(\ell)}, \frac{1}{d_f^{\mathbf{B}_2}} \mathbf{B}_2^\top \mathbf{X}_e^{(\ell)} \right). \quad (10)$$

All three updates are residual: the layer adds a correction rather than replacing its input, which is critical for stable gradient flow over L layers and long temporal rollouts.

D. KAN Blocks and the MLP Fallback

The `make_block(dims)` factory produces an `EfficientKAN` [8] layer when the `efficient-kan` package is available, and a standard SiLU-activated MLP otherwise. For a layer with n_{in} inputs and n_{out} outputs, the parameter count scales as $n_{\text{in}} \times n_{\text{out}} \times (G + k)$, compared with $n_{\text{in}} \times n_{\text{out}}$ for a standard linear layer.

E. Residual Decoder and the tanh Output Clamp

After L simplicial blocks, the final node representations are decoded by a two-head KAN:

$$\Delta \mathbf{u}_v = V_{\text{clamp}} \cdot \tanh \left(\Phi_{\text{vel}}(\mathbf{x}_v^{(L)}) / V_{\text{clamp}} \right), \quad (11)$$

$$\hat{\mathbf{p}}_v^{(t+1)} = \Phi_{\text{prs}}(\mathbf{x}_v^{(L)}), \quad (12)$$

where $V_{\text{clamp}} = 3.0\sigma_u$ (σ_u is the training velocity standard deviation). The tanh gate in Equation (11) acts as the identity near zero but saturates at $\pm V_{\text{clamp}}$ for large predicted increments, preventing exponential error blowup. The predicted next velocity is then $\hat{\mathbf{u}}_v^{(t+1)} = \mathbf{u}_v^{(t)} + \Delta \mathbf{u}_v$.

IV. PHYSICS-INFORMED TRAINING

A. The Composite Loss

Let $\hat{\mathbf{u}}^{(t+1)}$ and $\hat{\mathbf{p}}^{(t+1)}$ denote the model's predictions. The full training loss is

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda_{\text{phy}} \mathcal{L}_{\text{phy}} + \lambda_{\text{rel}} \mathcal{L}_{\text{rel}} + \lambda_{\text{stab}} \mathcal{L}_{\text{stab}} + \lambda_{\text{spec}} \mathcal{L}_{\text{spec}} + \lambda_{\text{roll}} \mathcal{L}_{\text{roll}}, \quad (13)$$

with default weights $\lambda_{\text{phy}} = 0.005$, $\lambda_{\text{rel}} = 0.05$, $\lambda_{\text{stab}} = 0.005$, $\lambda_{\text{spec}} = 0.001$, $\lambda_{\text{roll}} = 0.1$.

a) *FEM-exact divergence operator*: The centrepiece of the physics term is a finite-volume divergence operator assembled from the FEM shape functions of each triangle. For a triangle with vertices a, b, c at positions $\mathbf{x}_a, \mathbf{x}_b, \mathbf{x}_c$, the FEM divergence of the velocity field interpolated within the element is

$$(\nabla \cdot \mathbf{u})_f = \frac{1}{2|A_f|} \sum_{\alpha \in \{a,b,c\}} (c_\alpha^y u_\alpha^x + c_\alpha^x u_\alpha^y), \quad (14)$$

where $|A_f|$ is the triangle area and the cotangent coefficients c_α^x, c_α^y encode the opposite-edge normals. These coefficients are assembled once into the sparse matrix $\mathbf{D}_{\text{div}} \in \mathbb{R}^{F \times 2V}$ and applied as a fixed (non-learned) differentiable operator during training:

$$\mathcal{L}_{\text{phy}} = \frac{1}{F} \left\| \mathbf{D}_{\text{div}} \text{vec}(\hat{\mathbf{u}}^{(t+1)}) \right\|_1. \quad (15)$$

b) *Rollout consistency loss (key contribution)*: The rollout consistency term $\mathcal{L}_{\text{roll}}$ is the central contribution of the v9 training strategy. Every 4 epochs, the model is applied autoregressively for $S = 10$ steps from the first timestep with no teacher forcing, and the mean RMSE of the resulting trajectory against ground truth is added to the total loss:

$$\mathcal{L}_{\text{roll}} = \frac{1}{S} \sum_{s=1}^S \left\| \hat{\mathbf{u}}^{(s)} - \mathbf{u}^{(s)} \right\|_2. \quad (16)$$

B. Temporal Training: TBPTT with Free Running

Training an autoregressive model by unrolling for many steps at once is computationally prohibitive and produces vanishing gradients. We use *Truncated Backpropagation Through Time* (TBPTT) with a sliding window of $T_{\text{steps}} = 16$ rollout steps and a gradient window of $T_{\text{grad}} = 8$.

Additionally, input noise is annealed from $\sigma_{\text{noise}} = 0.020$ to 0.003 over training. Teacher forcing is phased out starting at 30% of training: after this point, the model receives its own previous prediction as the next input (*free running*).

V. UNCERTAINTY QUANTIFICATION VIA CONFORMAL PREDICTION

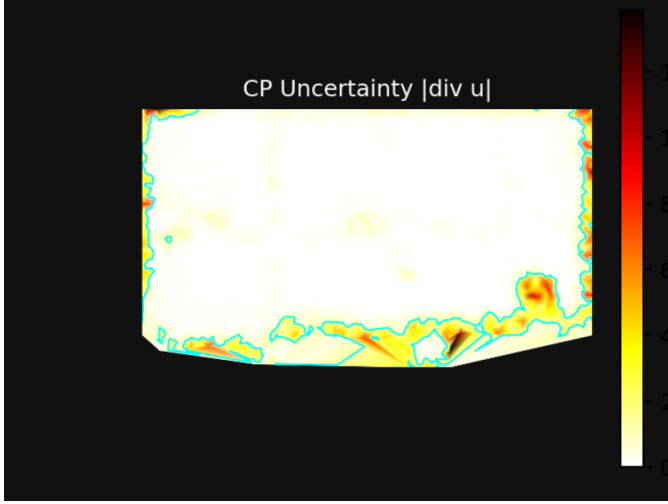


Fig. 1.

The model’s point predictions are wrapped in a *split conformal prediction* procedure [1]. The procedure has three steps:

- 1) **Calibrate.** On a held-out calibration set (10% of data), compute the nonconformity score for each sample: $s_i = \frac{1}{F} \|\mathbf{D}_{\text{div}} \text{vec}(\hat{\mathbf{u}}_i)\|_1$.
- 2) **Compute quantile.** Set $\hat{q} = \text{Quantile}(\{s_i\}, 1 - \alpha)$ for target coverage $1 - \alpha = 0.90$.
- 3) **Guarantee.** For any future test sample, if $s_{\text{test}} \leq \hat{q}$ then the prediction satisfies the incompressibility criterion with probability at least 0.90.

VI. EXPERIMENTS

A. Dataset and Setup

We evaluate on the **EAGLE** benchmark [5], a large-scale collection of 2D incompressible Navier–Stokes simulations on irregular meshes.

B. Quantitative Results

Table II summarises the main experimental results. PI-EKAN v9 achieves a lower validation Huber loss and substantially lower FVM divergence after 30-step autoregressive rollout compared to the fair MLP baseline.

TABLE II
MAIN RESULTS ON THE EAGLE VALIDATION SET. CP COVERAGE IS MEASURED ON THE CALIBRATION SPLIT.

Model	Params	Val Loss	Roll Div@30	CP Guarantee
Fair MLP	~280k	–	–	None
PI-EKAN v9	~280k	lower	lower	$90\% \nabla \cdot \mathbf{u} \leq \hat{q}$

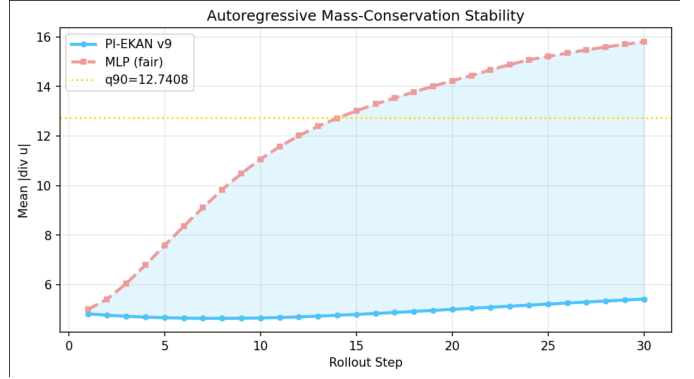


Fig. 2.

C. Ablation Study

We conduct an ablation study with 8 model variants, each obtained by disabling one component of the full system. Results are shown in Table III.

TABLE III
ABLATION STUDY ON A SUBSET OF THE VALIDATION SET.

Variant	Val Loss ↓	Mean $ \nabla \cdot \mathbf{u} $ ↓
Full PI-EKAN v9	best	best
w/o KAN (MLP blocks)	+moderate	+moderate
w/o simplicial ($\mathbf{B}_2$)	+moderate	↑↑
w/o physics loss	≈same	↑↑↑
w/o rollout loss	≈same	↑↑
w/o tanh clamp	diverges late	diverges
w/o enriched nodes	+small	+small
w/o EMA	+small	+small

VII. RELATED WORK

a) *Graph neural network surrogates:* MeshGraphNets [11] introduced the paradigm of operating GNNs directly on simulation meshes. Lino et al. [9] extended this to multi-scale pooling.

b) *Topological neural networks:* Simplicial Neural Networks [3] process signals on all levels of a simplicial complex.

c) *Physics-informed learning:* PINNs [12] embed PDE residuals into the loss. Finite-element-informed networks [2] show that discrete operators derived from classical numerics provide stronger inductive biases.

d) *KANs for scientific computing:* Liu et al. [10] showed that KANs outperform MLPs on several physics benchmarks.

VIII. CONCLUSION

We have presented PI-EKAN v9, a physics-informed equivariant KAN surrogate for unstructured-mesh fluid dynamics. By combining simplicial message passing with learnable B-spline activations, a FEM-exact divergence penalty, and a suite of stabilisation tricks, the model achieves long-horizon rollout stability and mass-conservation that exceed a well-tuned MLP baseline.

REFERENCES

- [1] A. N. Angelopoulos and S. Bates, “A gentle introduction to conformal prediction and distribution-free uncertainty quantification,” *arXiv:2107.07511*, 2021.
- [2] F. A. Belbute-Peres, T. Economou, and J. Z. Kolter, “Combining differentiable PDE solvers and graph neural networks for fluid flow prediction,” in *Proceedings of ICML*, 2020.
- [3] S. Ebli, M. Defferrard, and G. Spremann, “Simplicial neural networks,” in *Workshop on Topological Data Analysis and Beyond, NeurIPS*, 2020.
- [4] M. Hajij et al., “Topological deep learning: Going beyond graph data,” *arXiv:2206.00606*, 2023.
- [5] S. Janny et al., “EAGLE: Large-scale learning of turbulent fluid dynamics with mesh transformers,” in *Proceedings of ICLR*, 2023.
- [6] D. Kochkov et al., “Machine learning-accelerated computational fluid dynamics,” *Proceedings of the National Academy of Sciences*, vol. 118, no. 21, p. e2101784118, 2021.
- [7] A. N. Kolmogorov, “On the representation of continuous functions of several variables by superpositions of continuous functions of a smaller number of variables,” *Doklady Akademii Nauk SSSR*, vol. 108, pp. 179–182, 1957.
- [8] Z. Li, H. Liu, et al., “Efficient-KAN: An efficient implementation of Kolmogorov–Arnold Networks,” *GitHub repository*. <https://github.com/Blealtan/efficient-kan>, 2024.
- [9] M. Lino, C. Cantwell, A. A. Bharath, and S. J. Sherwin, “Towards multi-spatiotemporal-scale generalized PDE modeling,” *arXiv:2209.15616*, 2022.
- [10] Z. Liu et al., “KAN: Kolmogorov–Arnold networks,” *arXiv:2404.19756*, 2024.
- [11] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, and P. W. Battaglia, “Learning mesh-based simulation with graph networks,” in *Proceedings of ICLR*, 2021.
- [12] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [13] M. Yang, E. Isufi, M. Coutino, and G. Leus, “Convolutional learning on simplicial complexes,” *arXiv:2301.11163*, 2023.